

# Computer Vision Practical Report:

## AFFINE TRANSFORMATION

### 1. AIM

To read an input image using OpenCV, apply Affine Transformation using non-collinear reference points to alter the image's orientation and shape, display the original and transformed images side-by-side using Matplotlib, and save the result to disk.

### 2. PROCEDURE

- Load the input image and define three source points on it.
- Define three corresponding destination points to specify the new target locations.
- Calculate the  $2 \times 3$  affine transformation matrix using `cv2.getAffineTransform()`.
- Apply the transformation matrix to the image using `cv2.warpAffine()`.
- Display the original and transformed images side-by-side using Matplotlib.
- Save the output image to disk using `cv2.imwrite()`.

### 3. PROGRAM CODE

```
import cv2
import numpy as np

# Read the input image
img = cv2.imread("E11-IMG.webp")

# Check if image is loaded
if img is None:
    print("Error: Could not load image.")
    exit()

# Get image dimensions
rows, cols, ch = img.shape
```

```
# Define three points in the original image
pts1 = np.float32([
    [50, 50],
    [200, 50],
    [50, 200]
])

# Define corresponding points in the transformed image
pts2 = np.float32([
    [100, 100],
    [200, 50],
    [100, 250]
])

# Calculate the Affine Transformation Matrix
M = cv2.getAffineTransform(pts1, pts2)

print("Affine Transformation Matrix:")
print(M)

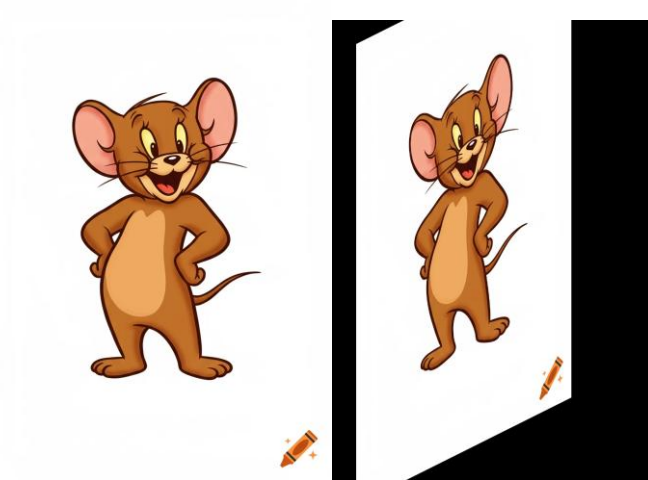
# Apply affine transformation
transformed = cv2.warpAffine(img, M, (cols, rows))

# Display original and transformed images
cv2.imshow("Original Image", img)
cv2.imshow("Affine Transformed Image", transformed)

# Save the transformed image
cv2.imwrite("affine_transformed.jpg", transformed)

# Wait for a key press and close windows
cv2.waitKey(0)
cv2.destroyAllWindows()
```

#### 4. OUTPUT



#### 5. RESULT

The Python script executed successfully. The input image was loaded.